

CUDA PROGRAMMING FOR NVIDIA H100S

cuTensorMap



Prateek Shukla · 简体中文版

TMA 是什么

TMA 是让数据搬运能够以最少的线程参与、完全异步进行的新单元。

单个线程发起 TMA 指令后立即继续执行，整个操作由硬件在后台处理。

你得到的是描述符，而不是地址计算。

TMA 可以同时向多个 SM 的共享内存传输数据。

能处理 1d-5d 张量。

H100 如何获得完美的异步性

我们需要所有单元随时保持忙碌。

H100 的正确做法是：用 TMA 让多个缓冲同时加载，并让张量核心（Tensor Core）同时执行运算。

更重要的一点是：你不需要大量复杂工程。描述符包办了从数据到布局再到优化的一切，把大量繁重工作留给硬件。

为什么我们需要描述符

描述符是拼图中让 H100 具备异步本性的一块。在更早的时候，整个数据获取都用索引完成：线程需要计算想要获取的数据的地址，然后才取回数据。这在 Ampere 上有所改善——拷贝指令是非阻塞的——但线程仍然要为每 16 字节计算地址。SM 仍被拴在拷贝引擎上。到了 H100，描述符封装了整个传输状态。由于完成这项工作所需的全部信息都包含在内存中那个 128 字节的对象里，硬件可以在后台完成任务，无需 SM 参与。

如何使用 **TMA** 进行拷贝

- 使用 CUDA API 创建一个 cuTensorMap
- 用所需的信息对它编码
- 发起该操作

cuTensorMap

它是一个描述符，存储关于以下内容的信息：

内存基址指针（设备地址）张量形状（每个维度的元素数量）步长（以字节计）数据类型对齐与 swizzle 内存空间（device、host 或 unified）顺序与 rank（最多 32 个维度）可选：tiling 或"交错"布局

创建 **tensormap** 对象

```
CUtensorMap tma_desc;
```

它在 host 内存中作为局部变量创建，大小为 128 字节，并且必须 128B 对齐。它不是普通指针，而是 NVIDIA 创造的一种特定数据结构。

cuTensorMapEncodeTiled() 函数用于把值编码进 tensormap。

CUtensorMap

这就是我们使用以下声明创建的 tensormap 对象：

`CUtensorMap tma_desc`，它将被填入编码后的描述符。

确定你要从 **HBM** 拷贝的实际数据的数据类型

TMA 引擎用它来自动获得内存对齐与传输大小。

数据类型

CU_TENSOR_MAP_DATA_TYPE_UINT8 - 无符号 8 位整数
CU_TENSOR_MAP_DATA_TYPE_UINT16 - 无符号 16 位整数
CU_TENSOR_MAP_DATA_TYPE_UINT32 - 无符号 32 位整数
CU_TENSOR_MAP_DATA_TYPE_INT32 - 有符号 32 位整数
CU_TENSOR_MAP_DATA_TYPE_UINT64 - 无符号 64 位整数
CU_TENSOR_MAP_DATA_TYPE_INT64 - 有符号 64 位整数
CU_TENSOR_MAP_DATA_TYPE_FLOAT16 - 16 位浮点（半精度）
CU_TENSOR_MAP_DATA_TYPE_FLOAT32 - 32 位浮点（单精度）
CU_TENSOR_MAP_DATA_TYPE_FLOAT64 - 64 位浮点（双精度）
CU_TENSOR_MAP_DATA_TYPE_BFLOAT16 - 16 位 brain 浮点
CU_TENSOR_MAP_DATA_TYPE_FLOAT32_FTZ - 带 flush-to-zero 模式的 32 位浮点
CU_TENSOR_MAP_DATA_TYPE_TFLOAT32 - TensorFloat-32 格式
CU_TENSOR_MAP_DATA_TYPE_TFLOAT32_FTZ - 带 flush-to-zero 模式的 TensorFloat-32

Flush-to-zero (FTZ) 模式是一种浮点算术优化，用于处理

非正规数（denormalized，即 subnormal）：用零替换它们，而不是正常地计算它们。

张量 **rank** 与全局内存地址

- 张量 rank 是张量的维度数量，不是线性代数中的矩阵秩
- 全局内存地址是张量在 HBM 中的内存地址

全局地址必须 16 字节对齐，以在硬件上获得高效的内存访问模式。

全局维度（**globalDim**）

一个数组，以元素数量（而不是字节）指定张量每个维度的大小。

对于一个 r 维数组，我们按以下方式组织数据：

`globalDim[0]` = 最内层维度

`globalDim[r]` = 最外层维度

注意该数组的每个元素取值范围是 0 到 2^{32} （约 40 亿）。

这些是每个维度中元素之间的字节步长

即硬件沿特定维度从一个坐标移动到下一个坐标时必须跳过多少字节。

最内层维度的步长是隐式的，由元素大小决定；该数组的大小为 $(\text{rank} - 1)$ 。

`globalStrides[0]` 是 `globalDim[1]`（次内层维度）的字节步长。

`globalStrides[rank-2]` 是 `globalDim[rank-1]`（最外层维度）的字节步长。

它是你想沿每个维度跳过的元素数量

即在执行异步拷贝的时候。

它也是一个大小为 rank 的数组，并且有特定要求：

它必须包含非零值。

步长值应小于或等于 8。

数组大小 = rank。

此外它以元素数量计算，而不是字节。

这是 **tile** 大小。它是一个大小为 **rank** 的数组，每个张量维度对应一个条目

与张量的 rank 相匹配。

它指定遍历 box 的大小（以元素计）——即每次 TMA 操作从全局内存传输到共享内存的那块数据。

以元素（而不是字节）为单位，与被传输的数据类型相对应。

内层维度必须 $\leq$ swizzle 大小。

共享内存

共享内存是附着在 SM 上的片上暂存存储 (scratchpad)。你的线程块获得其中一块区域，该块中的每个线程都可以读写该区域内的任意地址。在程序层面，它看起来像一个平坦的地址空间。

硬件在底层做的是把这块地址空间拆分成 32 个 bank。一个 bank 就是共享内存阵列中一条可独立服务的通道。之所以是 32 个，原因很简单：一个 warp 有 32 个线程，理想情况是每个线程占一个 bank。

要点：bank 不是 32 个由你手动索引的独立数组。你仍然只是计算一个地址，bank 由地址选出。

swizzle 与 bank 冲突

全部 32 个 bank 可以在一个周期内同时访问。一个 warp 发出一条逻辑上的加载/存储指令，但 SMEM 子系统可能分若干轮执行它：

第 1 轮：来自每个 bank 的请求的一个子集

第 2 轮：剩余的冲突请求，依此类推

然而，如果多个线程访问同一个 bank，访问就会串行化，造成 bank 冲突。在共享内存中访问数据的常规方式是 strided 访问模式，这会导致串行化。

1 个请求 -> 无冲突，2 个请求 -> 2 路冲突，4 个请求 -> 4 路冲突，8 个请求 -> 8 路冲突，32 个请求 -> 最坏情况冲突

swizzle 如何工作

它是一个地址映射函数。它取逻辑地址（你的程序以为自己在写的 GmemAddress），对它的位进行"洗牌"，生成物理地址（数据真正落地的 SmemAddress）。

我们需要把顺序访问模式分散到所有 bank 上，以避免冲突。

为此我们有 3 种 swizzle 模式：32B、64B、128B。

128B、64B 与 32B 布局定义了内存 swizzle 的跨度（即块大小），从而也决定了内存事务的大小。根据你应用程序的内存访问模式选择正确的布局，是关键的性能调优步骤。

bank 方程

其中 a' 是硬件实际使用的共享内存字节地址。如果启用了 swizzle, a' 就是 swizzle 后的地址, 而不是未 swizzle 的逻辑地址。这个 32 bank、4 字节步长的模型是推理冲突的标准方式, 也与你之前使用的地址位视图一致。

这意味着 bank ID 来自地址位 $a'[6:2]$:

然后它每 $32 * 4 = 128$ 字节重复一次。

swizzle 公式

$$a' = a \wedge ((a \& Y_mask) \gg 3)$$

其中 a 是字节地址， Y_mask 是 $1 \ll 7$ ，且低 4 位 $a[3:0]$ 永远不被触碰。32B: $a[4]' = a[4] \text{ xor } a[7]$
64B: $a[5:4]' = a[5:4] \text{ xor } a[8:7]$ 128B: $a[6:4]' = a[6:4] \text{ xor } a[9:7]$ 被 XOR 的那些位是共享内存中 swizzle 模式行索引的低位，本质上并不是你数学矩阵的行。只有当你的布局把矩阵行映射到那些共享内存模式行上时，它们才成为你的矩阵行位。

swizzle 跨度与 atom

对于 SM90，完整的 swizzle 模式每以下字节数重复一次：

32B swizzle : 256 字节

64B swizzle : 512 字节

128B swizzle : 1024 字节

每行 128 字节，每行被拆成 8 个 16 字节的 cell。swizzle 在行与行之间置换这 8 个 cell。所以 $\text{repeat_bytes} = 128\text{B} * \text{distinct_rows}$ 。

swizzle 行不是你的矩阵行。它是一条 128 字节的共享内存行。swizzle 只置换那些 16B 块。在 swizzle atom 内部，元素不是被随机打乱的——硬件/布局保持小的连续组完好无损，只是把这些组作为单元进行置换。

128B swizzle

Span = 128 字节, Atom = 16 字节硬件把目的地的共享内存区域看作：一行行 128 字节的行，每行拆成 8 个 16 字节的 cell；行内 cell 索引： $x = 0..7$ ，行索引： $y = 0, 1, 2, \dots$ 如果共享内存基地址对齐到 swizzle 的完整模式边界，逻辑 cell (y, x) 的物理目的地是： $\text{phys_addr} = \text{base} + 128*y + 16*x$ 我们像这样应用 swizzle 公式： $a[6:4]' = a[6:4] \wedge a[9:7]$ 整个 128B 行被视为 8 个可 swizzle 的 16B cell。行号对 8 取模来选择置换，模式每 8 行重复一次。总重复大小 = $8 * 128B = 1024B$

64B swizzle

64B swizzle 作用于由 4 个 16B cell 组成的组。行内 cell 索引： $x = 0..3$ 行索引： $y = 0, 1, 2, \dots$ 逻辑 cell (y, x) 的物理目的地是： $\text{phys_addr} = \text{base} + 128*y + 16*x'$ $x' = x \wedge (y \& 3)$ $a[5:4]' = a[5:4] \wedge a[8:7]$ 行的左 64B 半边按行号 mod 4 做置换，右 64B 半边以同样方式置换。模式每 4 行重复一次，总重复大小 = $4 * 128B = 512B$

32B swizzle

32B swizzle 作用于成对的 16B cell。行内 cell 索引： $x = 0-1$ 行索引： $y = 0, 1, 2, \dots$ 逻辑 cell (y, x) 的物理目的地是： $\text{phys_addr} = \text{base} + 128*y + 16*x'$ $x' = x \oplus (y \& 1)$ $a4' = a[4] \oplus a[7]$ 在每个 32B 区域内，两个 16B cell 每隔一个 128B 行交换一次。完整模式每 2 行重复一次。由于每行是 128B，重复周期为 256B

CUDA PROGRAMMING FOR NVIDIA H100S

设置 swizzle

Prateek Shukla · 简体中文版

交错

全局内存中的数据并不总是线性的。像 cuDNN 这样的库主要使用 NCHW 内存布局，以在卷积和其他操作中获得最佳 GPU 性能。

CUtensorMapInterleave 参数告知 TMA 如何解码地址空间。它把 HBM 中物理上交错的排布映射为共享内存中逻辑上线性的重建。

Hopper TMA 支持为 NC/xHWCx 布局设计的特定交错模式：CU_TENSOR_MAP_INTERLEAVE_16B

CU_TENSOR_MAP_INTERLEAVE_32B

两种主要模式

布局：NC/8HWC8（8 通道的向量）。

数学：8 通道 $\times$ 2 字节（FP16）= 16 字节。

行为：数据以 16 字节交错的块访问。

布局：NC/16HWC16（16 通道的向量）。

数学：16 通道 $\times$ 2 字节 = 32 字节。

行为：数据以 32 字节交错的块访问。

注意：如果通道数不是分片（slice）的倍数，最后一个分片必须零填充，以维持交错粒度。

交错与 **swizzle**

耦合：交错与 swizzle 并非相互独立。

要求：文档声明：

"当 `interleave` 为 `CU_TENSOR_MAP_INTERLEAVE_32B` 时，`swizzle` 参数必须设置为 `CU_TENSOR_MAP_SWIZZLE_32B`。"

理由：从全局内存去交错 32B 块的硬件路径，直接接入共享内存中 32B atom 的 swizzle 逻辑。两者同步联动，把复杂的全局布局映射为无 bank 冲突的共享内存布局。

几个要点

如果使用 32B 交错，你的全局地址应当 32B 对齐；如果使用 16B 交错，全局地址应当 16B 对齐。

此外，如果使用 16B 交错，全局步长应是 16 的倍数；如果使用 32B 交错，全局步长应是 32 的倍数。

此外，如果你使用交错，维度数必须大于或等于 3。

CUDA PROGRAMMING FOR NVIDIA H100S

用法



Prateek Shukla · 简体中文版

L2 提升

与从 L2 缓存取数据相比，从 HBM 取数据很慢。

预取（prefetch）通过发出从 HBM 到 L2 缓存的非阻塞传输，来预判未来的数据需求。

不提升时（`CU_TENSOR_MAP_L2_PROMOTION_NONE`），内存控制器使用标准的缓存行抓取大小（通常 32 字节）。这效率不高，因为 L2 缓存以 128 字节行为单位管理数据。

把抓取大小提升到 128B 或 256B 后，单个 TMA 请求就能保证一次把更大的连续数据块带入 L2 缓存。

这对 GEMM 这类可预测模式最大化带宽效率，确保数据恰好在计算单元需要时驻留在 L2 中。

CUDA PROGRAMMING FOR NVIDIA H100S

L2 提升枚举及其硬件含义

Prateek Shukla · 简体中文版

CU_TENSOR_MAP_L2_PROMOTION_L2_64B 与 CU_TENSOR_MAP_L2_PROMOTION_NONE

CU_TENSOR_MAP_L2_PROMOTION_NONE：使用默认的 32 字节 sector 抓取。如果你的张量非常稀疏或步长巨大，抓取邻居数据就是浪费。用它来避免"过度抓取"（把带宽浪费在无用的数据上）。

CU_TENSOR_MAP_L2_PROMOTION_L2_64B：抓取两个相邻的 32B sector（共 64 字节）。

对内层维度恰好为 64 字节（32 个元素）的特定半精度（FP16）张量形状有用。

CU_TENSOR_MAP_L2_PROMOTION_L2_128B

一次抓取完整的 128 字节缓存行。

把 DRAM 命令开销减少 75% (1 条命令代替 4 条)。

对齐：与 CUDA 的标准向量加载大小匹配 ($\text{ld.global.v4} = 16\text{B} \times 32 \text{ 线程} = 512\text{B}$ ，通常拆成 128B 块)。

建议：这是大多数稠密 FP16/BF16 GEMM 操作的标准默认选择。

CU_TENSOR_MAP_L2_PROMOTION_L2_256B

在单个逻辑事务中抓取两条完整缓存行（256 字节）。

H100 的内存控制器足够健壮，可以处理多行预取。

要求数据密度高。你的应用程序必须立即消费这些数据，才配得上占用的缓存空间。

如果数据没被使用，缓存污染的风险最高。

一个小而重要的提示

经验法则：

稠密 GEMM/卷积：始终使用 L2_128B 或 L2_256B。最大化总线效率。

嵌入查找 / 稀疏：使用 NONE (32B)。不要抓取你不会触碰的数据。

调优变量：选择取决于 Tensor_Inner_Dim_Bytes。

如果内层维度 $< 64B$, L2_128B 可能是浪费。让提升大小匹配你的连续数据宽度。

oobFill

oobFill 帮助处理 H100 GPU 上张量拷贝操作中的越界情况。

在目的地自动用零或 NaN 填充越界区域，而不是在 HBM 中。

HBM 中的源数据在整个过程中保持不变。

免除 kernel 中的手动边界检查，提升代码清晰度与性能。

对 tiling 操作很有用，尤其是 tile 延伸到张量边缘之外时。

填充的选择取决于应用需求；padding 用零，调试用 NaN。

用法

第二条语句使用 NaN，但在 FMA 操作期间它被当作零处理。